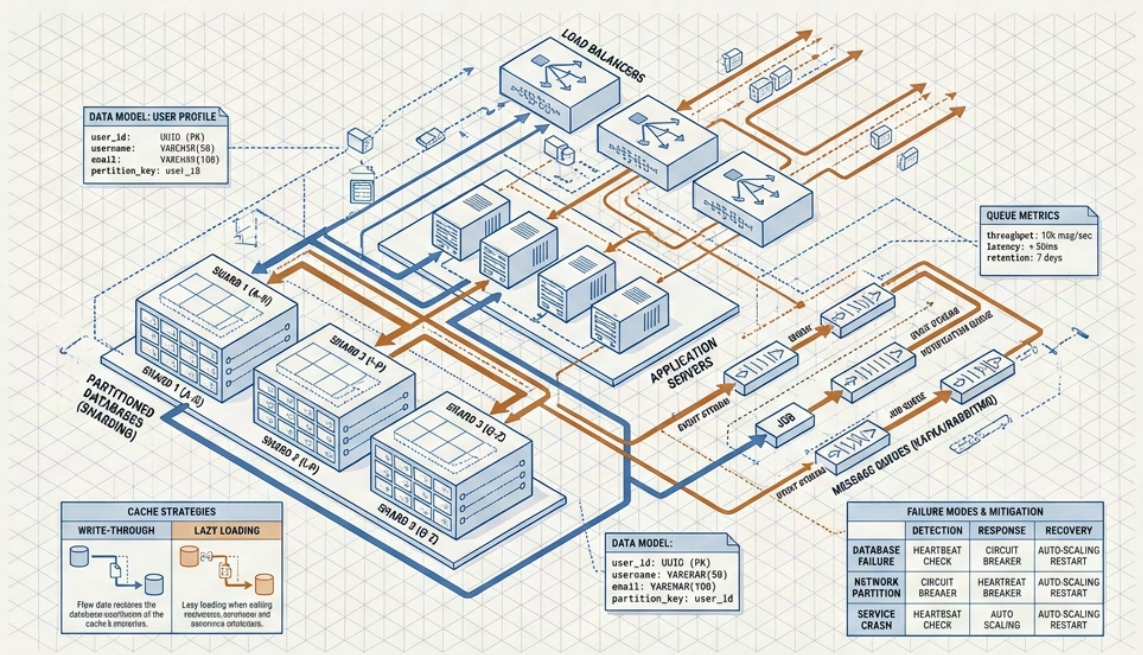


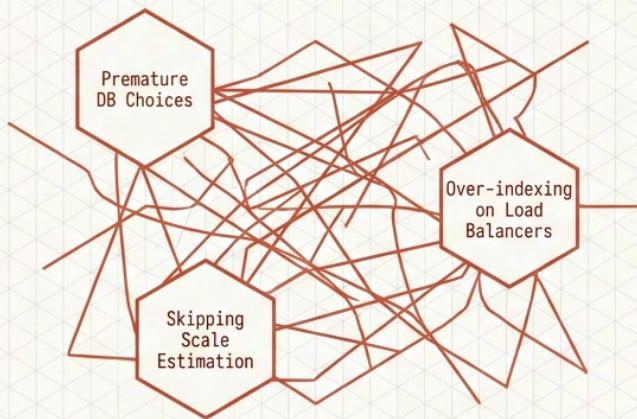
The 45-Minute System Design Master Plan

An Architect-Level Interview Framework

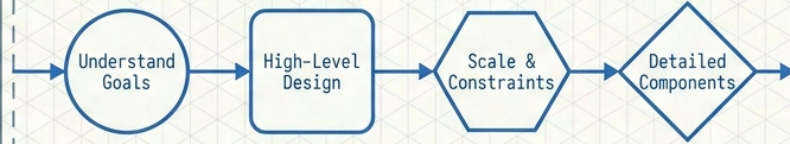


The Chaos of an Unstructured Interview

The Trap



The Framework



Without a framework, you spend 45 minutes discussing low-value components and run out of time before addressing the critical technical challenges. The goal is mastering a repeatable mental process, not memorizing architectures.

The Hidden Rubric

What the interviewer is actually measuring.

Structuring Ambiguity



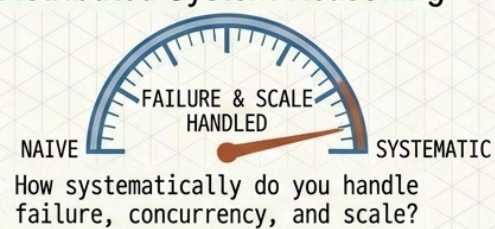
Defending Decisions



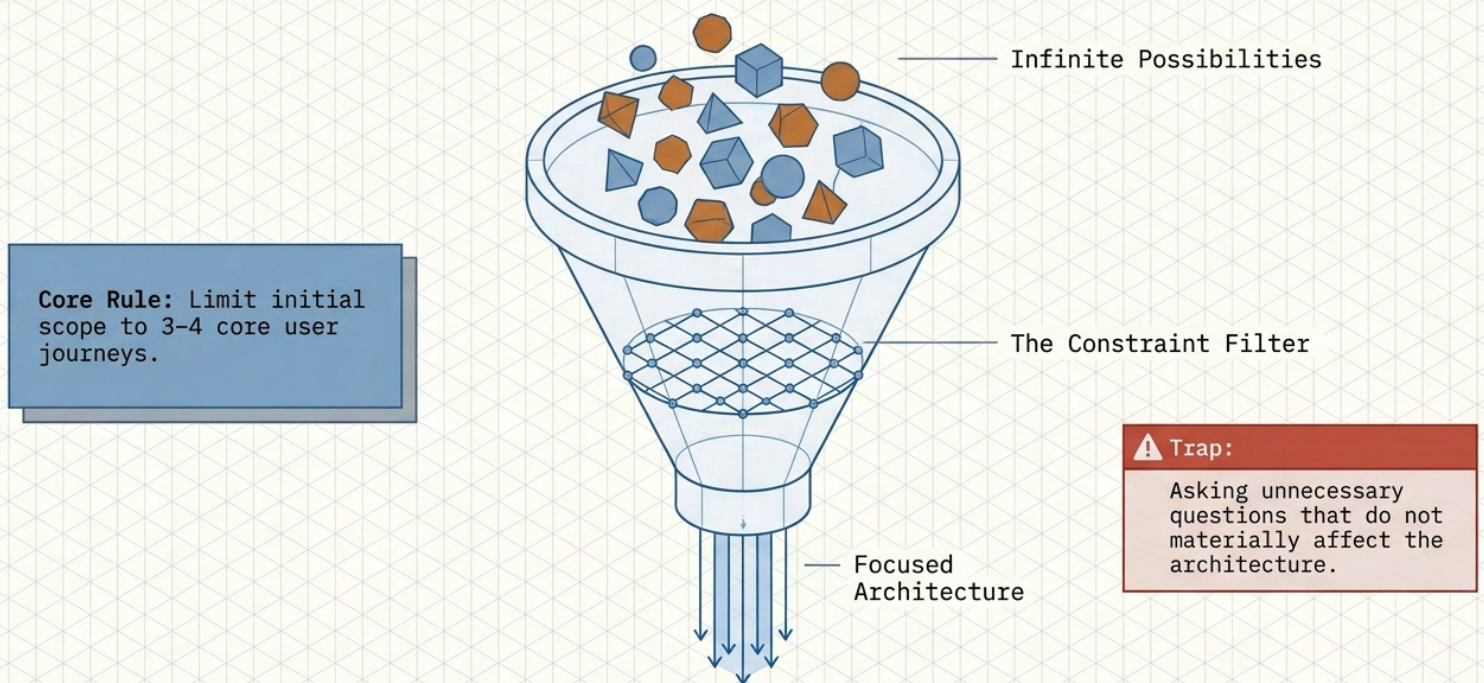
Prioritization



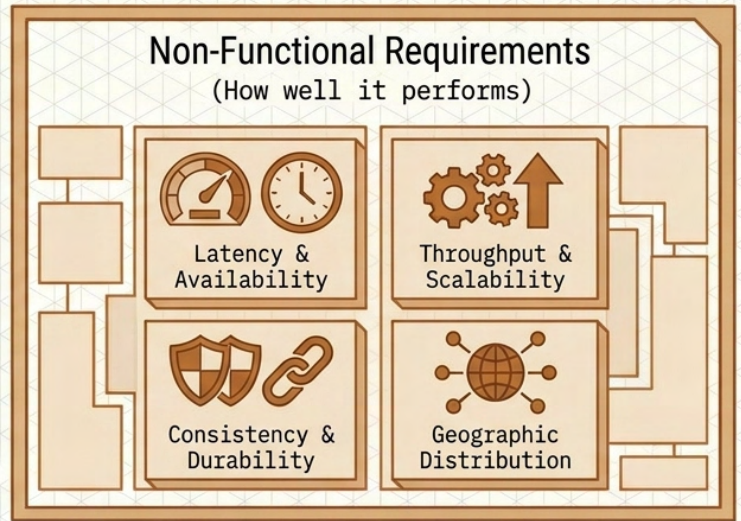
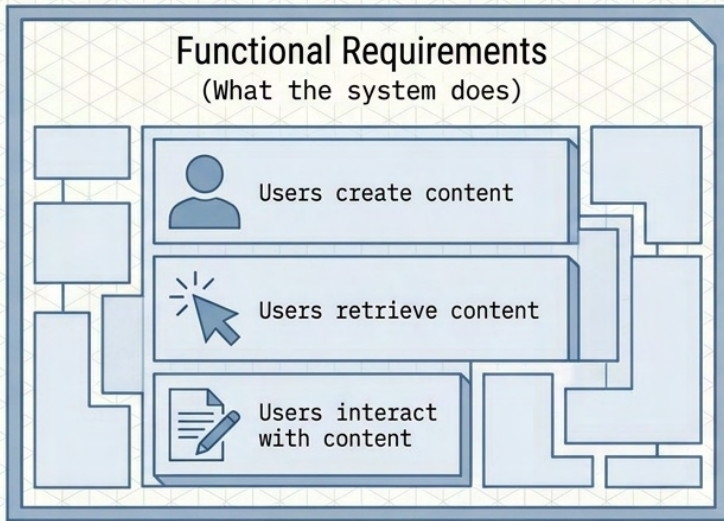
Distributed System Reasoning



Phase 1: Signal Over Noise



The Constraint Filter



Why do NFRs matter? Geographic distribution dictates CDN and replication strategies. The read/write ratio ratio dictates your caching strategy.

Applied Example: Railway Reservation System

Search

Find trains by source, destination, and date.



Availability

Show real-time seat inventory by class and quota.



Reservation

Select passengers and temporarily hold eligible inventory.



Payment & Confirmation

Initiate payment; guarantee seat is never double-booked.

Core Non-Functional Constraints:

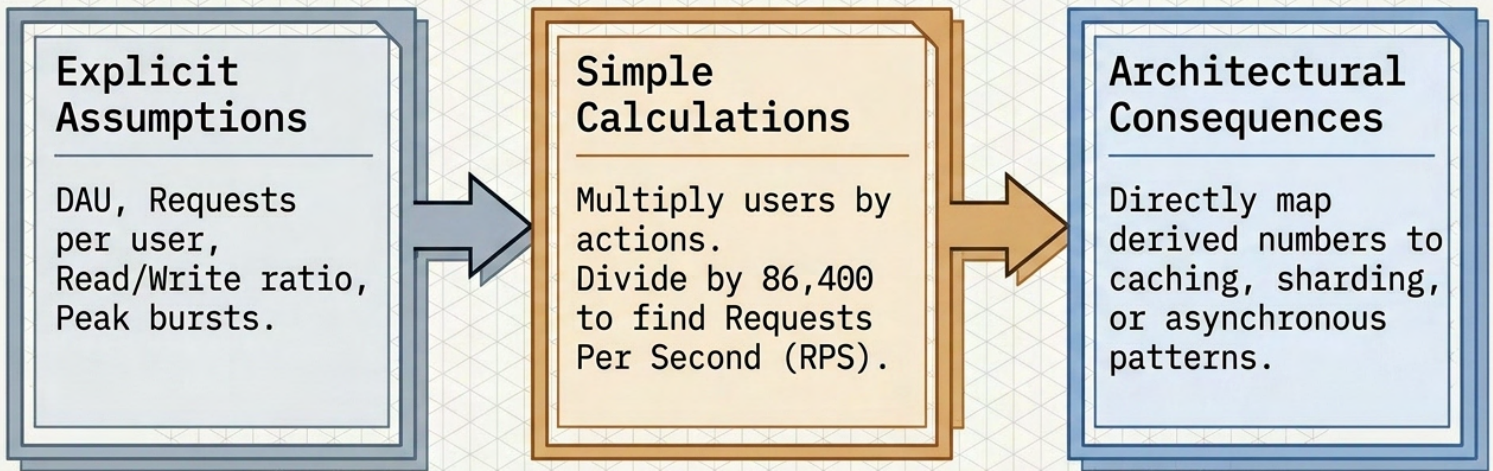
Target 99.99% Availability

Search p95 < 500 ms

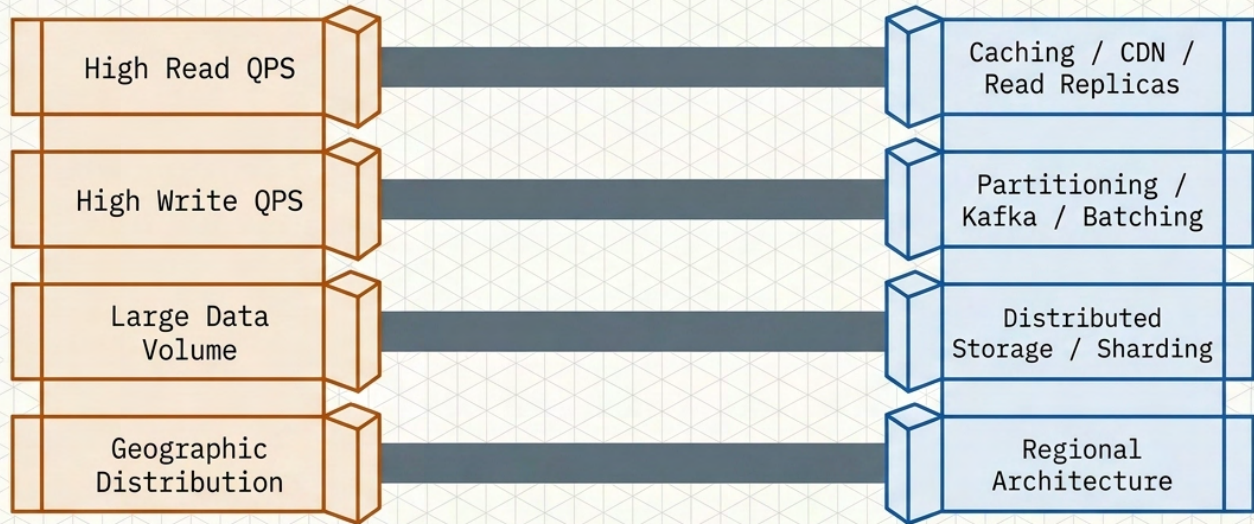
Strong Consistency for Allocation

Phase 2: Math With a Purpose

Trap: Inventing random numbers that have no bearing on your design.

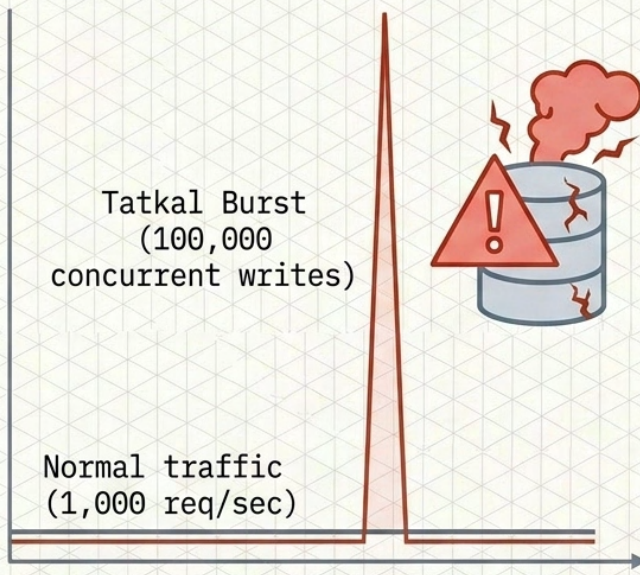


The Metric-to-Architecture Bridge



Never calculate a number without attaching it to a design choice.

Applied Scale: The 100k Burst



Assumptions:

Normal traffic: 1,000 req/sec

Peak booking traffic: 100,000 writes/sec

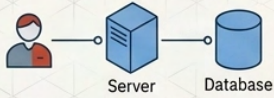
Read/Write ratio: Extremely read-heavy

Consequence: The primary database will melt. We must cache read-heavy data, partition hot inventory, inventory, and keep the strongly consistent write path as small as possible.

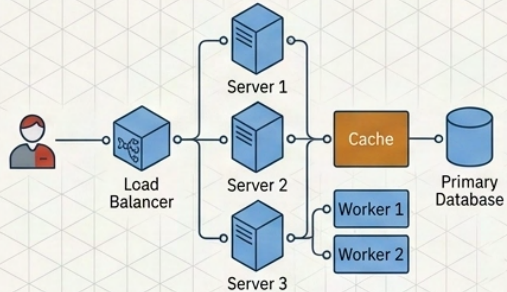
Phase 3: Incremental Complexity

Trap: Drawing a massive 15-component microservices web immediately.

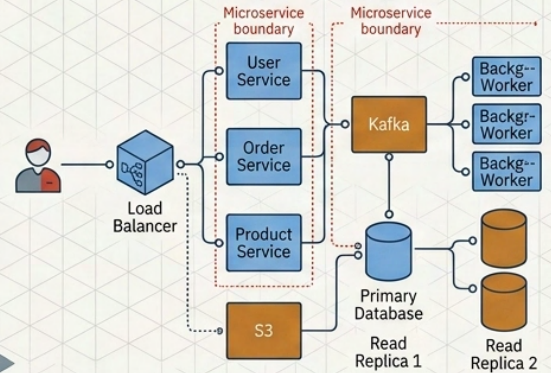
Single User Baseline



Medium Cluster

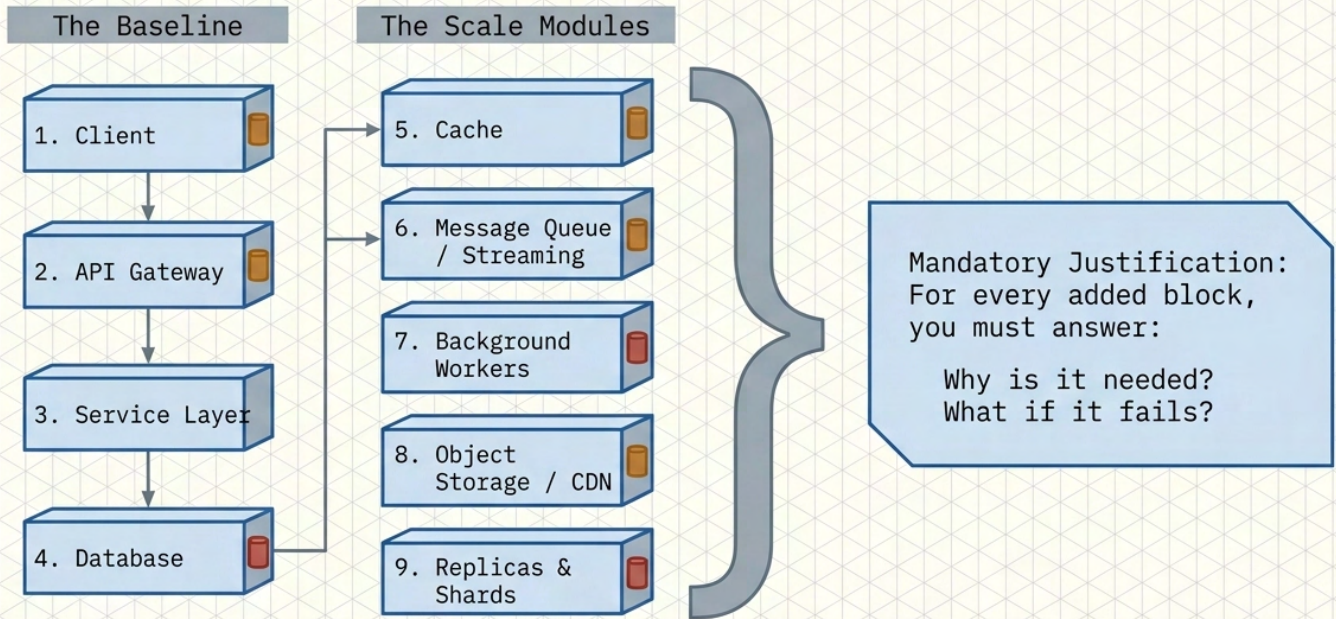


Distributed Web

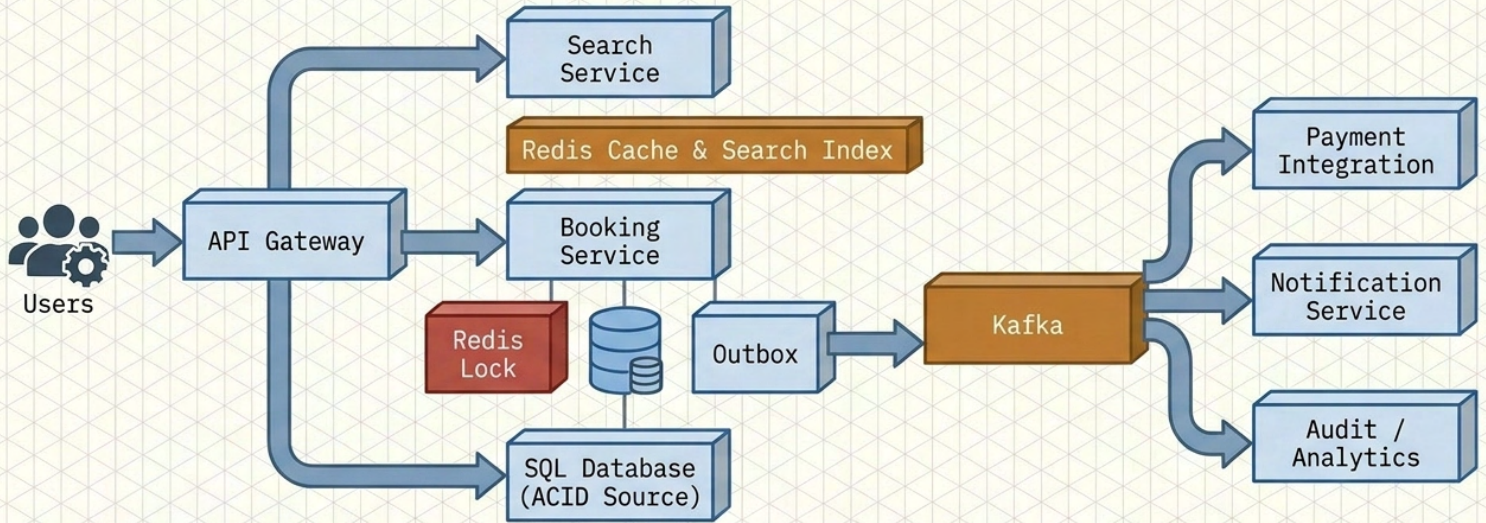


Core Rule: Start with a single user. Every added component must solve a specific scale problem.

The Architectural Build Order



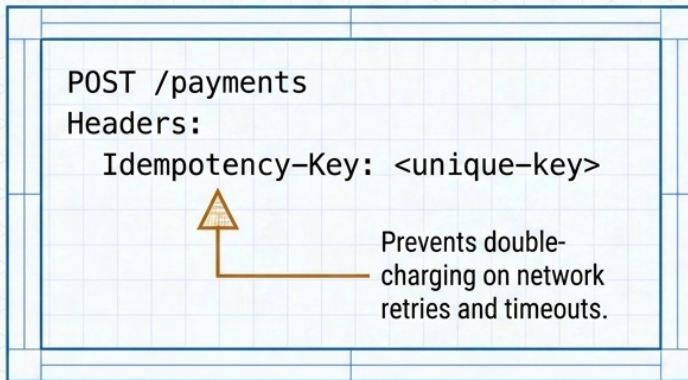
Applied Architecture: Railway High-Level Design



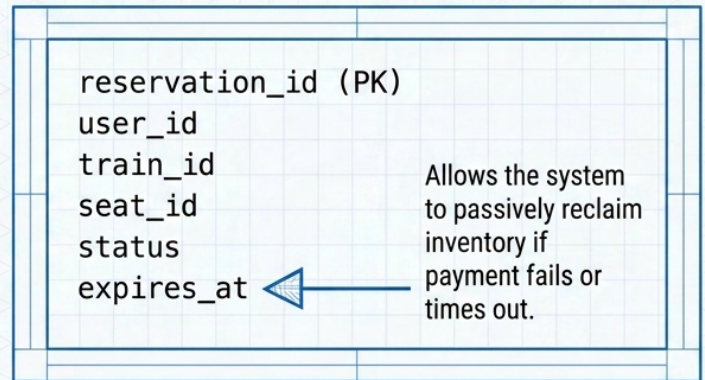
Architectural Principle: Redis coordinates and accelerates; the SQL Database enforces correctness; Kafka decouples asynchronous work.

Anatomy of Core APIs & Data Models

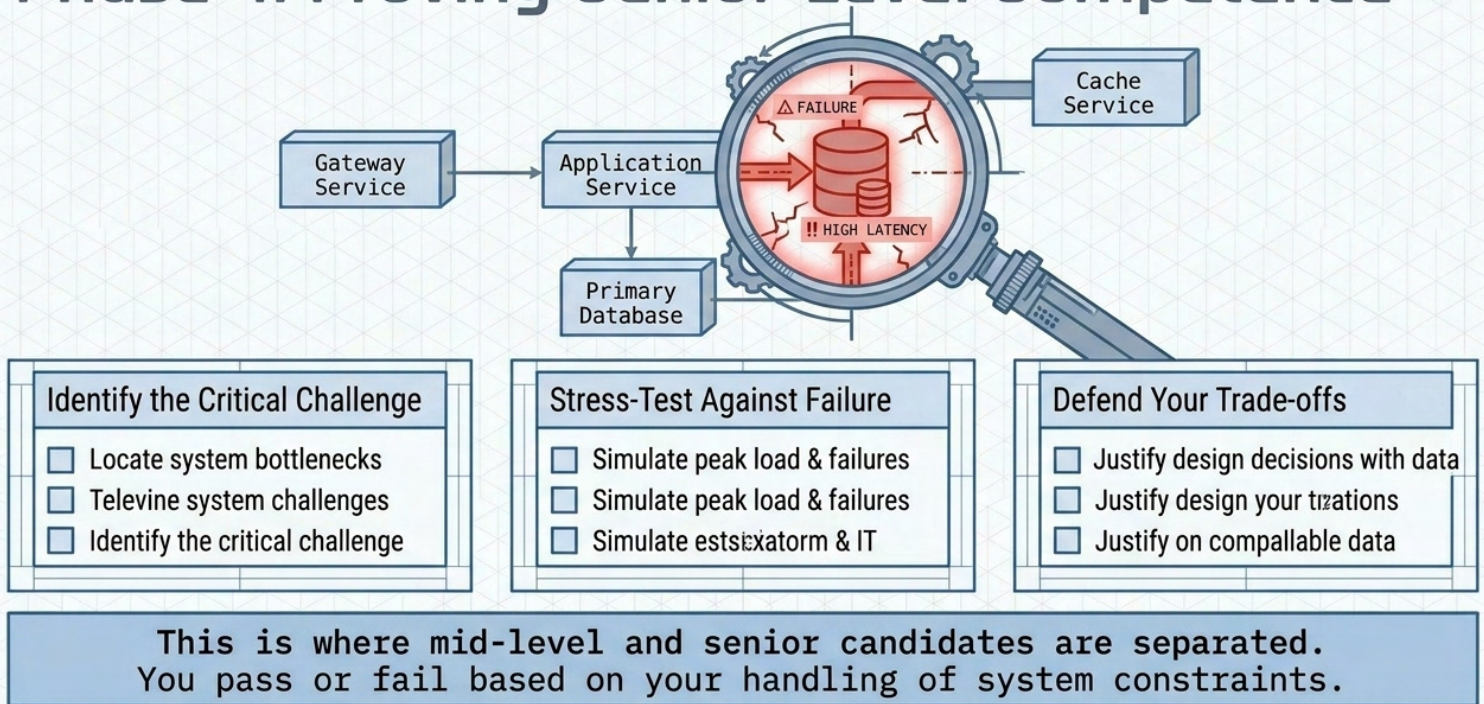
API Anatomy: /payments



Data Model Anatomy: reservation table



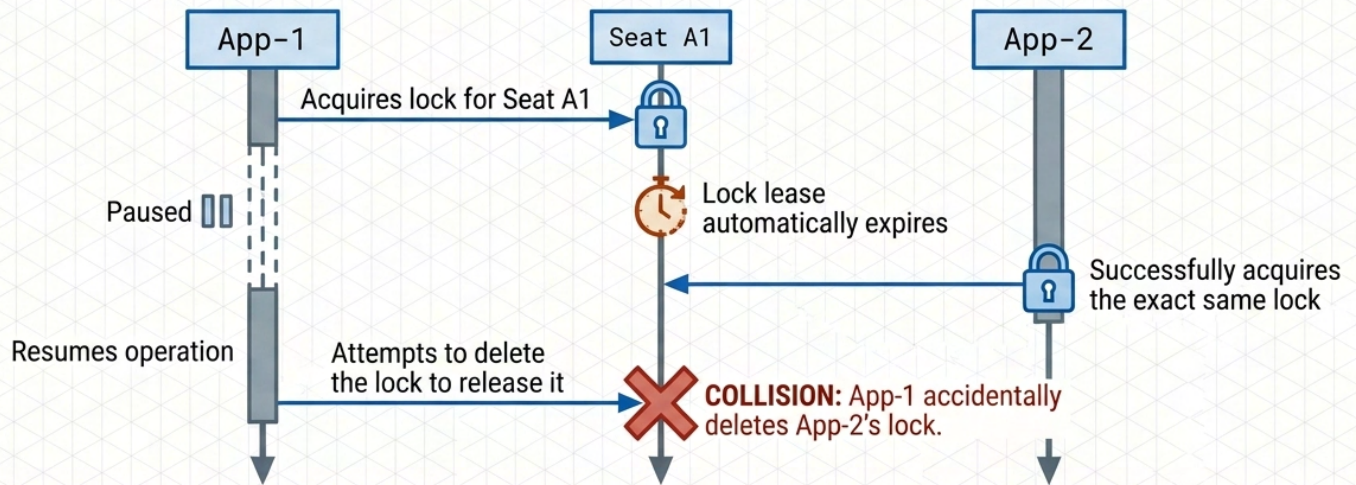
Phase 4: Proving Senior-Level Competence



The Critical Challenge: Double Booking

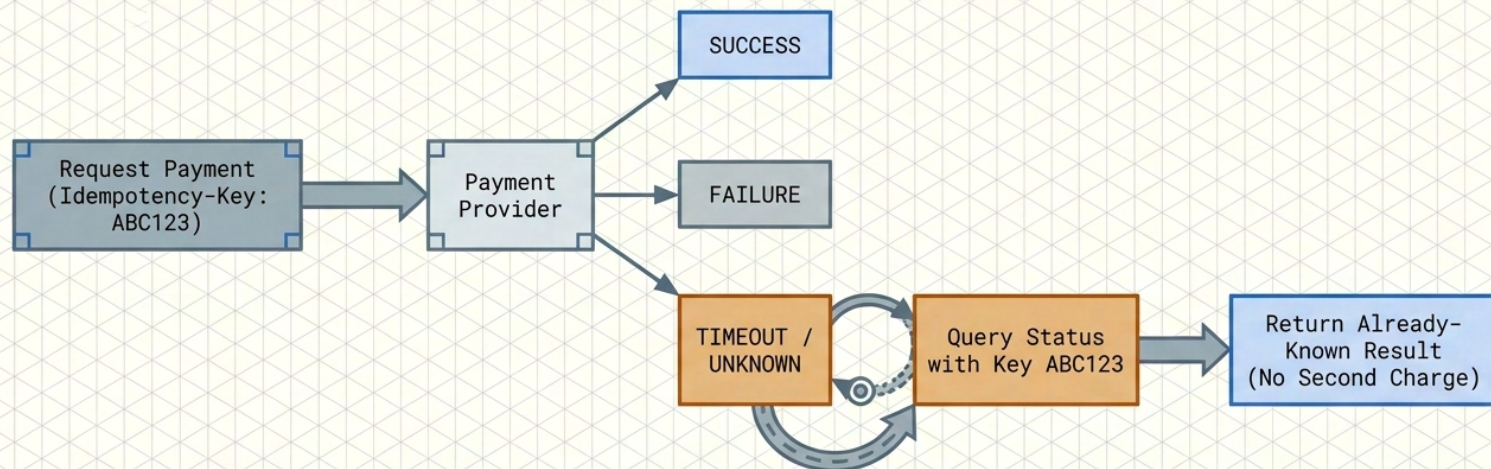
Distributed Coordination Matrix		
Strategy	Mechanism	Verdict in Distributed System
JVM Synchronized Lock	Coordinates threads inside a single application instance.	Fails in distributed deployment. Does not lock other instances. 
Redis Distributed Lock	Centralized in-memory lock using SET NX PX.	Reduces contention and is fast, but vulnerable to lease expiry if processes pause. 
DB ACID Transaction	Relies on database uniqueness and constraint checks.	The final correctness boundary. Mathematically safe, but slow under heavy contention. 

Visualizing Lock Failure & Partitioning



- Solution: The lock must have an owner identity (UUID). Deletion must use an atomic Lua script to verify ownership before release.
- Partitioning: Don't lock the global bottleneck (lock:train:12623). Lock the optimized partition (lock:train:12623:coach:A1).

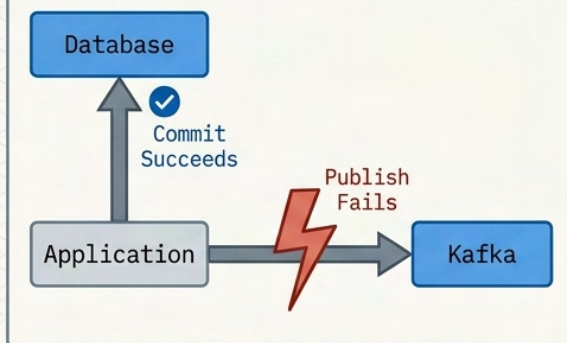
The Payment Trap & Idempotency



Core Rule: External payment systems create distributed transaction boundaries. Never assume a timeout means a failure. Use the **Idempotency-Key** to safely query and retry without duplication.

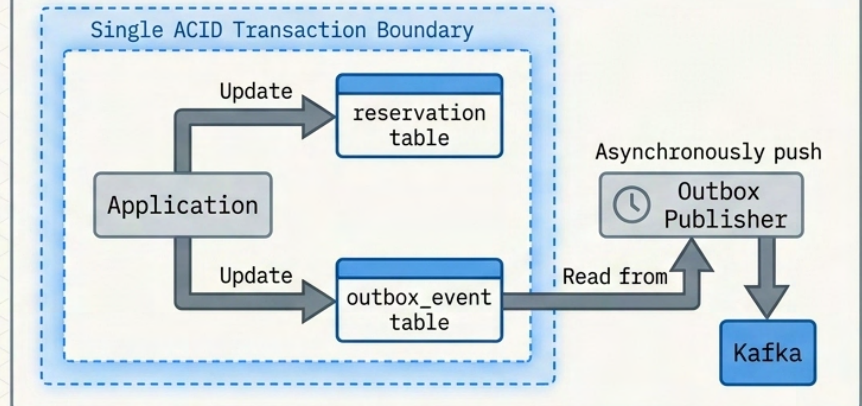
Solving the Dual-Write Problem

The Trap: Dual Write



Data divergence.

The Solution: Outbox Pattern



Principle: Use the Outbox pattern when a database state change and a downstream event must not diverge.

Failure Matrix & Observability

Failure Response Matrix

Component	Failure State	Required Architectural Response
Redis	Crashes	Database remains the ultimate correctness boundary.
Kafka	Unavailable	Outbox retains events safely until cluster recovers.
Payment	Timeout	Query provider status via Idempotency Key.
App Instance	Crash after Hold	Passive expiry process reclaims held reservations.

Observability Checklist

 Track p95 API Latency

 Reservation Hold Expiry Rates

 Kafka Consumer Lag

 Redis Lock Contention

The Architect Mindset

1. Why does this exist?

Every component must solve a specific problem.

2. How does it scale?

Scale by partitioning. Avoid global locks.



Leadership

3. What happens when it fails?

Correctness first. Ensure safe retries and data durability.

4. What trade-offs did you accept?

Decouple asynchronously. Accept eventual consistency where safe.

The 45-Minute Framework: Cheat Sheet

